

Test Plan

Project: Cebuana Lhuillier Parking Reservation System

Version: 2.0

Date: February 24, 2026

Related: [Test Scripts](#) | [Test Results](#) | [SRS](#)

1. Overview

This document defines the testing strategy, scope, approach, and test environment for the Cebuana Lhuillier Parking Reservation System. The system serves three user roles (End User, Admin, Attendant) and must be validated across functional, integration, UI, performance, and security dimensions.

2. Test Objectives

Objective	Description
Functional Correctness	Verify all user stories from the BRD are implemented correctly
Data Integrity	Ensure no double-bookings, proper status transitions, accurate counts
Security Validation	Confirm authentication, authorization, and data protection controls
Usability	Verify mobile responsiveness and role-appropriate UX flows
Performance	Validate API response times and page load speeds
Regression	Ensure new features do not break existing functionality

3. Scope

3.1 In Scope

Area	Details
Authentication	Login, logout, session management, password change (with current password verification), forgot password, session timeout

Area	Details
End User Flows	Vehicle CRUD (with plate uniqueness), booking flow, multi-date booking, hourly timeline view, cancellation, QR code, notifications, waitlist join/leave
Admin Flows	User CRUD (including job_family field), bulk upload (with job_family), bulk modify, building/floor/slot management, zone management, parking config (including waitlist settings and main_building_exclusive toggle), reports, AI insights, building policies (sticker/vehicle validation, slot registration, open floors), site customization, event blocking (create/list/delete) , admin cancellation with mandatory reason + user notification
Attendant Flows	Daily reservations view, confirm arrival, report no-show (with <code>no_show_at</code> timestamp), QR scan
Business Rules	Zone restrictions, external booking rules, VIP privileges, no-show date guard, waitlist expiry rules, plate number uniqueness, vehicle-level sticker/slot validation, main building exclusivity , event block double-booking prevention
API Endpoints	All 70+ REST endpoints across 16 route modules
Background Tasks	Auto no-show marking (with <code>no_show_at</code> timestamp), slot release (based on <code>no_show_at</code>), waitlist expiry
Database	Index effectiveness, data consistency, migration scripts
Security	DOM password exposure prevention (CWE-312), HttpOnly cookies, RBAC, rate limiting

3.2 Out of Scope

Area	Reason
Email notifications	Not yet implemented
CSV/Excel report exports	Not yet implemented
Load testing (>1000 users)	Requires dedicated infrastructure
Third-party AI API reliability	Dependent on external service

4. Test Approach

4.1 Test Levels

Level	Description	Tools
Unit Testing	Individual functions and API endpoint handlers	pytest, pytest-asyncio
Functional Testing	End-to-end API flows with real database	curl, httpx, pytest
Integration Testing	Cross-module interactions (auth + booking + notifications)	pytest, Playwright
UI Testing	Frontend rendering, navigation, form validation	Playwright (browser automation)
Performance Testing	API response times, concurrent request handling	curl timing, async load scripts
Security Testing	Authentication bypass, injection, header validation	curl, manual penetration testing

4.2 Test Types

Type	Coverage
Positive Tests	Happy path — expected inputs produce expected outputs
Negative Tests	Invalid inputs, unauthorized access, boundary conditions
Boundary Tests	Max dates (7), empty lists, long strings, special characters
Regression Tests	Re-run existing tests after each feature addition

5. Test Environment

Component	Specification
Backend	FastAPI on Python 3.11+, running on port 8001
Frontend	React 19 SPA, running on port 3000
Database	MongoDB 7.x (local instance)
Browser	Chromium (Playwright headless)
OS	Linux (Kubernetes container)

Component	Specification
API Base URL	{REACT_APP_BACKEND_URL}/api

5.1 Test Accounts

Role	Email	Password	Notes
Admin	admin.test@cebuana.com	Test123!	Full system access
User	user.test@cebuana.com	Test123!	Standard employee
Attendant	attendant.test@cebuana.com	Test123!	Parking attendant
Policy User	lgdeguzman@pjlhuillier.com	Test123!	Test user for building policy (Pacific Star)

6. Test Data Strategy

Data Type	Strategy
Users	Seeded via admin account on startup; additional users created via bulk upload
Buildings	Created through admin UI/API; test with 2-3 buildings, 2 floors each
Vehicles	Created per test user; at least 1 vehicle per booking test user
Reservations	Created during test execution; verified via API and UI
Zones	Created with building assignments; tested with user zone restrictions

7. Entry & Exit Criteria

7.1 Entry Criteria

- Application deployed and accessible at test URL
- All test accounts available and functional
- Database seeded with minimum test data (admin user)
- Backend and frontend services running without errors

7.2 Exit Criteria

- All P0 test cases pass (100%)

- P1 test cases pass (95%+)
- No critical or high-severity bugs open
- All security test cases pass
- Performance benchmarks met (API < 500ms p95)

8. Risk Matrix

Risk	Probability	Impact	Mitigation
Session timeout during long test sequences	Medium	Medium	Use fresh login before each test suite
Test data pollution between test runs	Medium	High	Clean up test data after each suite; use unique identifiers
Rate limiting blocks test execution	Low	Medium	Space login attempts; use pre-authenticated sessions
AI insights tests fail due to API key	Medium	Low	Skip AI tests if key not configured; test error handling path
Background task timing affects test results	Medium	Medium	Account for 5-minute cycle; use direct API calls for validation

9. Test Execution Schedule

Phase	Test Types	Duration	Dependencies
Phase 1: Core API	Unit + Functional (Auth, Users, Buildings)	1 day	Backend running
Phase 2: Booking Flow	Functional + Integration (Reservations, Slots, Zones)	1 day	Test data seeded
Phase 3: Admin Flows	Functional (Bulk upload, Config, Reports)	1 day	Buildings + Users created
Phase 4: Attendant	Functional + Integration (Daily view, No-show, QR)	0.5 day	Active reservations
Phase 5: UI/UX	UI automation (all pages, mobile responsive)	1 day	All backend tests pass
Phase 6: Security	Security validation (headers, auth bypass, injection)	0.5 day	Full application running

Phase	Test Types	Duration	Dependencies
Phase 7: Performance	Response time benchmarks	0.5 day	Stable environment

10. Defect Classification

Severity	Definition	Example	SLA
Critical	System crash, data loss, security breach	Double-booking, auth bypass	Immediate fix
High	Major feature broken, no workaround	Cannot book, cannot login	Fix within 4 hours
Medium	Feature partially broken, workaround exists	Filter not working, wrong count	Fix within 1 day
Low	Cosmetic, minor UX issue	Alignment, tooltip text	Fix in next release

Next: [Test Scripts](#) | [Test Results](#)